EXPERIMENT NO. 3 A (Group A)

Aim: Implementation of Support Vector Machines (SVM) for classifying images of handwritten digits into their respective numerical classes (0 to 9).

Name:Aditya Rade Roll.No:BEAD23167

Outputs:

Step 1 — Import libraries

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
```

Step 2 — Load digits

In [2]:
```
digits = load_digits()

X = digits.data
y = digits.target

print("Dataset shape:", X.shape)
print("Target shape:", y.shape)
```

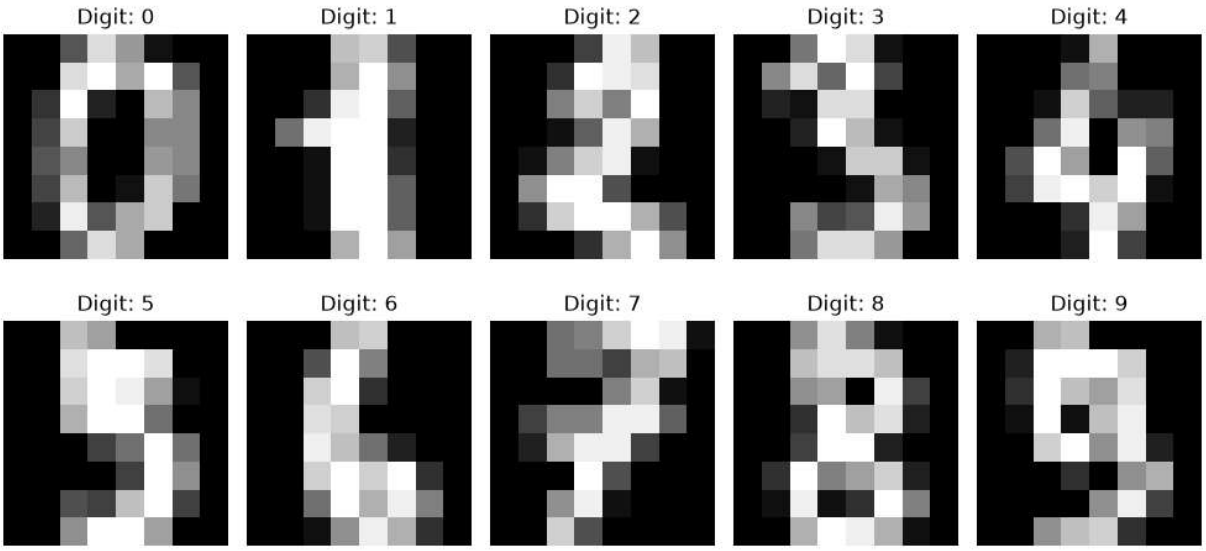
```
Dataset shape: (1797, 64)
Target shape: (1797,)
```

Step 3 — Display digits

In [3]:
```
fig, axes = plt.subplots(2, 5, figsize=(10, 5))

for ax, image, label in zip(
    axes.ravel(),
    digits.images[:10],
    digits.target[:10]
):
    ax.imshow(image, cmap="gray")
    ax.set_title(f"Digit: {label}")
    ax.axis("off")

plt.tight_layout()
plt.show()
```



Step 4 — Split dataset

In [4]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 1437
Testing samples: 360
```

Step 5 — Train SVM

In [5]:
```
model = SVC(
    gamma=0.001,
    C=100
)

model.fit(X_train, y_train)

print("SVM model trained successfully.")
```

```
SVM model trained successfully.
```

Step 6 — Prediction

In [6]:
```
y_pred = model.predict(X_test)

print("First 20 predictions:")
print(y_pred[:20])

print("\nActual values:")
print(y_test[:20])
```

```
First 20 predictions:
[5 2 8 1 7 2 6 2 6 5 0 5 9 3 4 4 2 4 9 9]

Actual values:
[5 2 8 1 7 2 6 2 6 5 0 5 9 3 4 4 2 4 9 9]
```

Step 7 — Accuracy

In [7]:
```python
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Accuracy percentage:", accuracy * 100)
```

```
Accuracy: 0.9916666666666667
Accuracy percentage: 99.16666666666667
```

Step 8 — Confusion Matrix

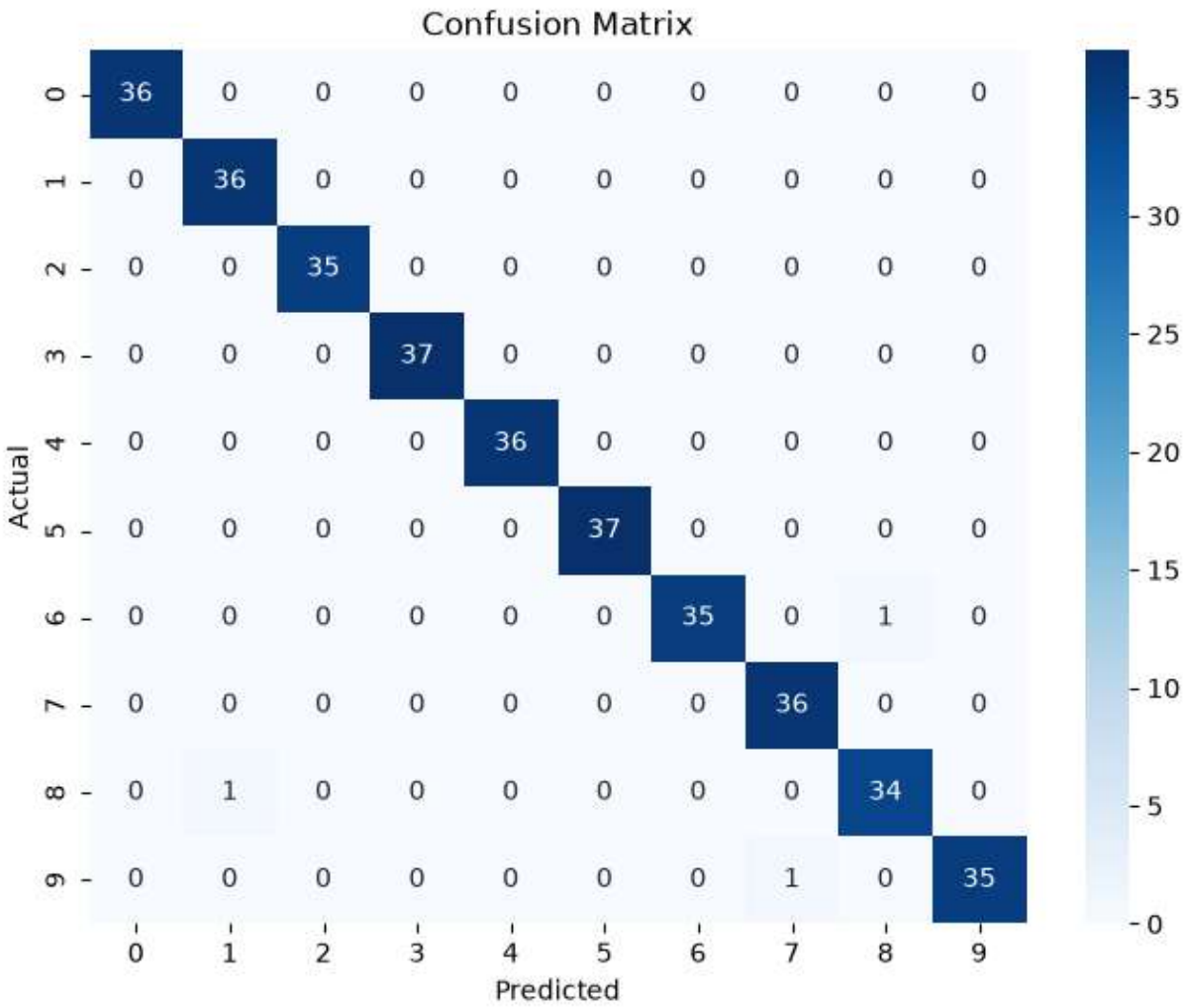
In [10]:
```python
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



Step 9 — Classification report

In [12]:
```
print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Name:Aditya Rade Roll No:BEAD23167")
```

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        36
           1       0.97      1.00      0.99        36
           2       1.00      1.00      1.00        35
           3       1.00      1.00      1.00        37
           4       1.00      1.00      1.00        36
           5       1.00      1.00      1.00        37
           6       1.00      0.97      0.99        36
           7       0.97      1.00      0.99        36
           8       0.97      0.97      0.97        35
           9       1.00      0.97      0.99        36

    accuracy                           0.99       360
   macro avg       0.99      0.99      0.99       360
weighted avg       0.99      0.99      0.99       360

Name:Aditya Rade Roll No:BEAD23167
```